

Object Oriented Programming

UML (Unified Modeling Language) ↷

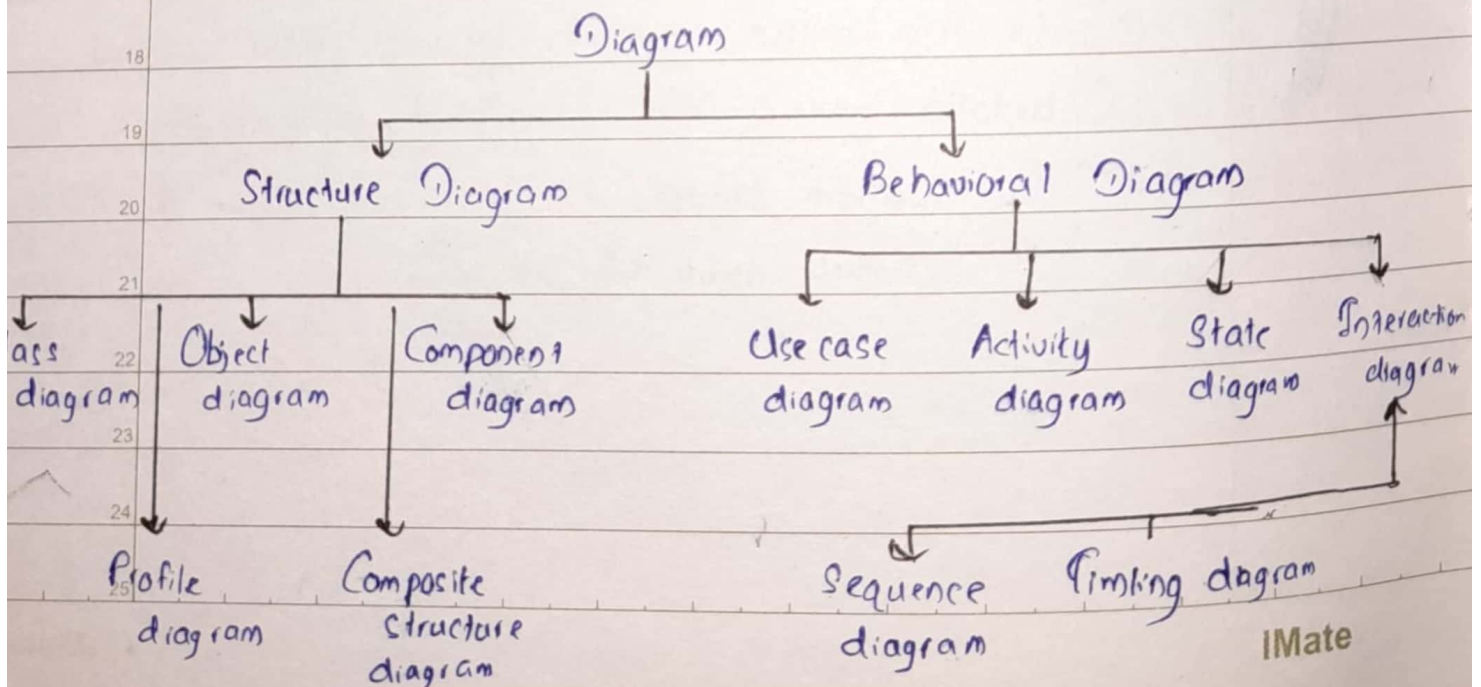
* UML is a pictorial language used to make a software blueprints.

The building blocks UML can be defined as,

- (1) Things
- (2) Relationships
- (3) Diagram

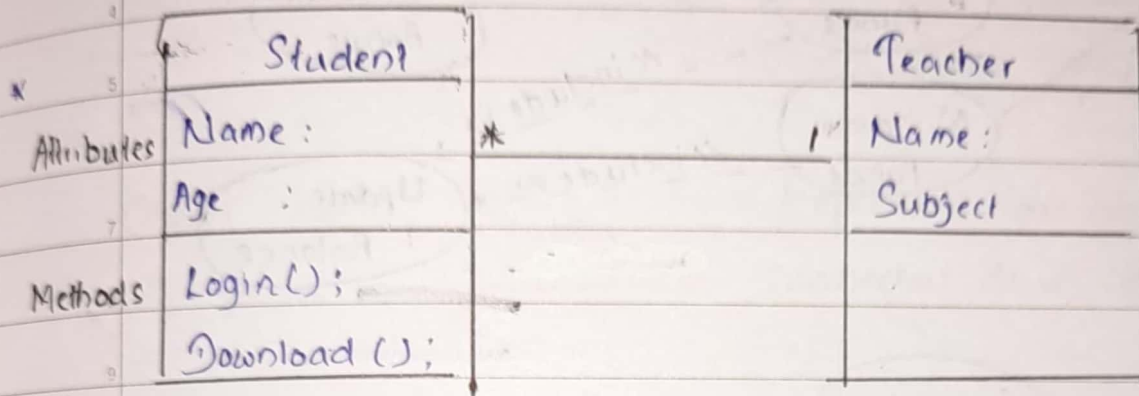
Things can be,

- * Structural
- * Behavioral
- * Grouping
- * Annotational

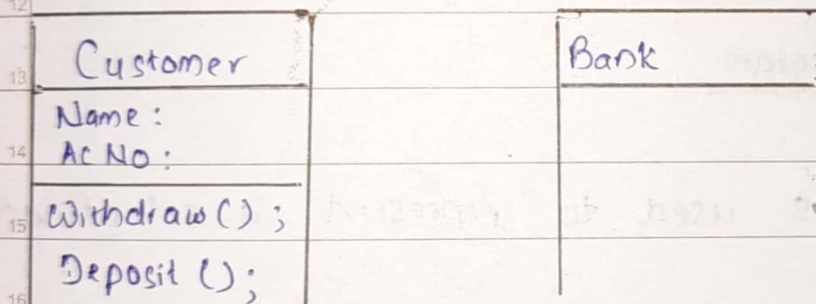


Class diagram

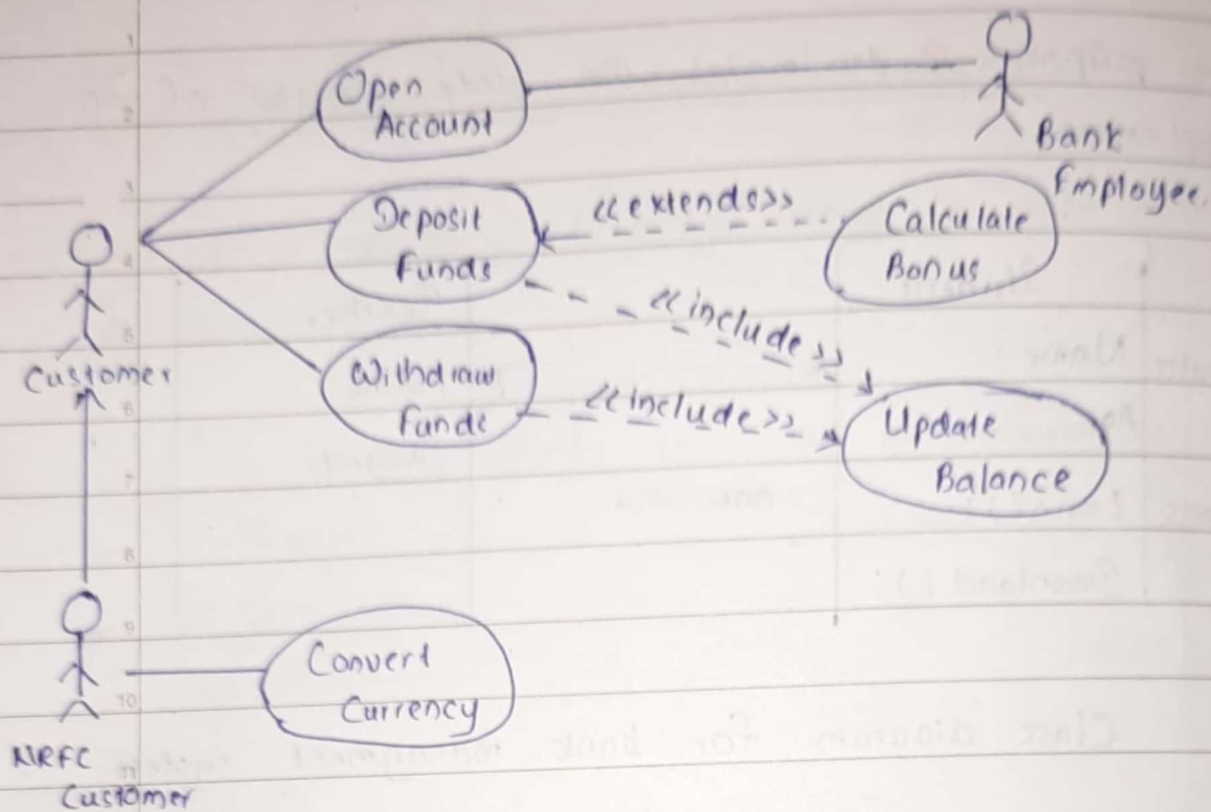
- * The purpose is to model the static view of an application.



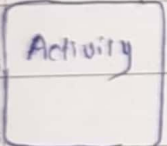
Class diagram for bank management system

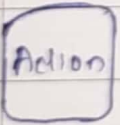


Use case diagram

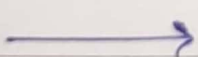


Activity Diagram

Activity  is used to represent a set of actions

Action  A task to be performed.

Control flow  Shows the sequence of execution.

Object flow  Show the flow of an object from one activity (or action) to another activity

Initial Node



Portrays the beginnings of set of actions or activities.

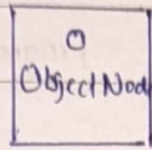
Object Flow

Activity Final Node



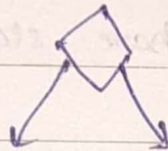
Stop all control flows and Object flows in any activity.

Object node



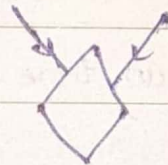
Represent an object that is connected to a set of objects.

Decision node

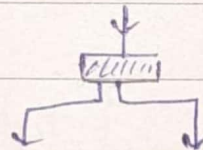


Represent a test condition to ensure that the control flow or object flow only goes down one path.

Merge Node

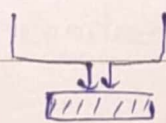


Fork Node



Split behavior into a set of parallel or concurrent flows of activities.

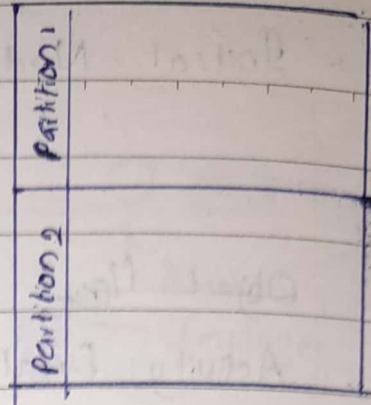
Join Node



Bring back together a set of parallel or concurrent flows of activities.

Swimlane and Partition

* A way to group activities performed by the same actor on an activity diagram or to group activities in a single thread.



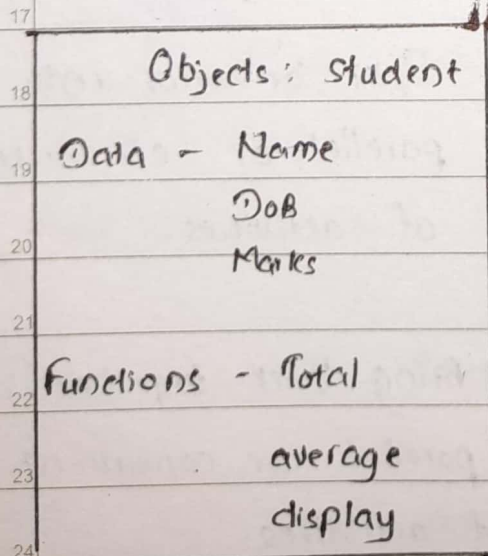
Object Oriented Programming

* OOP is a style of computer programming which represent concepts as objects that have state and behavior.

properties (or state : Characteristics that can change)

Behavior (or abilities : Things they can do)

* Objects are the basic run time entities in an object-oriented system.



Classes

Date

No

* It is a blueprint that defines the variables and the methods common to all objects of a certain kind.

* A class is used to manufacture or create objects.

```
class className {
```

```
    # fields
```

```
    # Methods
```

```
}
```

Constructor

* A constructor in java is a special method that is used to initialize objects. The constructor is called when an object of a class is created.

* It has the same name as that of the class does not have any return type.

```
class Test {
```

```
    Test () {
```

```
        // Constructor body
```

```
    }
```

```
}
```


Types of Constructor,

1) No-arg constructor

2) Parameterized constructor

3) Default Constructor

1) No-argument Constructor

* If a constructor doesn't accept any parameters, it is a no-arg constructor.

```
ex:- private Constructor () {  
    //  
}
```

2) Parameterised Constructor

* A constructor can also accept one or more parameters.

```
ex:- Class Main {  
    String languages
```

```
    Main (String lang) {
```

```
        this.languages = lang;
```

```
    }
```

3) Default Constructor

* If we do not create any constructor, java automatically create a no-arg constructor during the execution of a program.

class Main {

Main() {

}

}

Important notes on Constructors

- * A java constructor must not have a return type.
- * The name of the constructor should be the same as the class.
- * A constructor cannot be abstract, static or final.
- * A constructor can be overloaded but cannot be overridden.

Method overloading in Java

* If a class have multiple methods having same name but different parameters, it is Method overloading.

2 ways of overload the method in java.

- (1) By changing no of arguments.
- (2) By changing the data type

(1) class addNum {
 int add (int a, int b)
 { return a+b; }
 int add (int a, int b, int c)
 { return a+b+c; }

```

(9) class addNum {
    int add (int a, int b)
    { return a+b; }
    double add (double a, double b)
    { return a+b; }
}

```

* Constructor എന്നു ന്നു constructor എന്ന് call ന്നതാണ്
 this keyword ന്നു use ന്നതാണ് ഇത്.

```

class Main () {
    int a
    public static void Main ( ) {
        this (b,c); / this (s,e);
    }
    Main (int b, int c) {
        a = b+c;
    }
}

```

Method Overriden

* If a sublc subclass provides specific implementation of
 the method that has been declared by one of its
 parent class, it is known as method overriding.

— Usage of method overriding —

- * Used to provide the specific implementation of a method which is already provided by its superclass.
- * Used for runtime polymorphism.

```
ex: class Vehicle {  
    void run()  
    { SysOut ("Vehicle is running"); }  
}  
class Bike extends Vehicle {  
    void run()  
    { SysOut ("Bike is running safely"); }  
}
```

Java Access Modifiers

- * Access modifiers are used to set the accessibility (visibility) of classes, interfaces, variable, methods, constructors, data members, and the setter methods.

Default - Declarations are visible only within the package (package private)

Private - Declarations are visible within the class only.

Protected - Visible within the package or all subclasses.

Public - Visible everywhere.

Modifier	class	Package	Subclass	Global
public	yes	yes	yes	yes
protected	yes	yes	yes	No
Default	yes	yes	No	No
Private	yes	No	No	No

Java Inheritance

x It is one of the key features of OOP that allows us to create a new class from an existing class.

* The new class that is created is known as subclass and the existing class from where the child class is derived is known as super class.

x The extend keyword is used to perform inheritance in java.

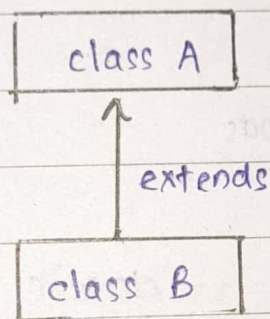
```
class Animal {
}
class dog extends Animal {
}
```

- * The most importance of inheritance is reusability. The code that is present in the parent class can be directly used by the child class.

Types of Inheritance

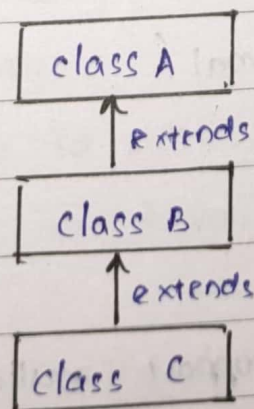
—— (1) Single Inheritance ——

- * A single subclass extends from a single superclass.



—— (2) Multilevel Inheritance ——

- * A subclass extends from a superclass and then the same subclass act as a superclass from another class



```
class Animal {
```

Date

No

```
}
```

```
class Dog extends Animal {
```

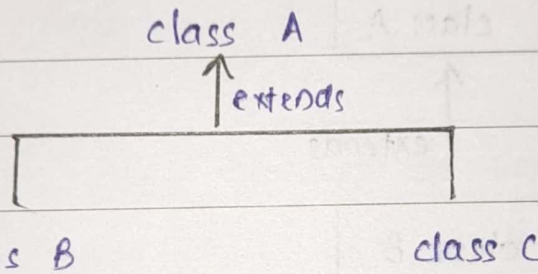
```
}
```

```
class Dog extends Animal {
```

```
}
```

———— (3) Hierarchical Inheritance ————

* Multiple subclasses extend from a single super class.



ex: class Animal {

```
}
```

```
class Dog extends Animal {
```

```
}
```

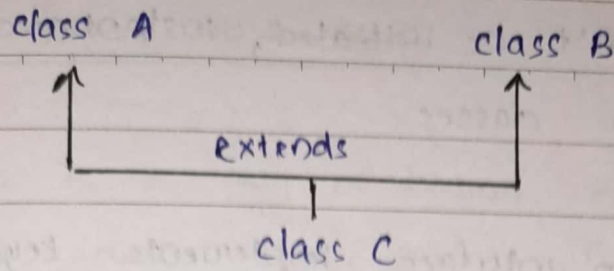
```
class Cat extends Animal {
```

```
}
```

———— (4) Multiple Inheritance ————

* Java doesn't support multiple inheritance.

We can achieve multiple inheritance using interfaces.



```

ex-1
class A {
}
class B {
}
class C extends [A and B] {
}
  
```

Java Interface

- * Interface is a blueprint of a class.
- * It has static constants and abstract methods.
- * This is a mechanism to achieve abstraction.
- * There can be only abstract methods in the java interface not method body.
- * This is used to achieve abstraction and multiple inheritance in java.

```

ex-1
interface <Interface Name> {
    // declare constant fields
    // " abstract methods
    // by default
}
  
```

* An interface can't be initiated, instantiated, it can be implemented by classes.

* To implement an interface, implements keyword is used.

(*) A class can implements multiple interfaces.

(*) An interface can extends multiple interfaces.

* We can't create variables within interface without initializing

ex:- String name = "Java"; ✓
String name; ✗

* We can't create constructors within interface.

because interface not have any instance fields
so nothing to construct.

Marker

* Marker / Tagging Interface - An interface with no methods is known as marker interface.

ex:- interface Animal {
}

(*) Why marker interface is ~~su-su~~ used?

It is used for better readability of code.

ex:- Serializable, Cloneable

Java Abstraction

- * Abstraction is a way of hiding complexity.
- x Data abstraction is the process of hiding certain details and showing only essential detail to the user.
- x Abstraction can be achieved with either abstract class or interfaces.
- x Abstract class : is a restricted class that cannot be used to create objects. (to access it, must be inherited from another class)
- * Abstract method : can only be used in abstract class, and it doesn't have a body. (The body is provided by the subclass (inherited from))

There are 2 ways to achieve abstraction in java.

1) Abstract class (0 to 100%.)

```
ex :- Abstract class test {  
    // method 1 () ← abstract methods  
    // method 2 () { ← non abstract methods  
        }  
    // method 3 ()  
}
```


Q2) Interface (100%).

```
ex- Interface A {  
    // method 1()  
    // method 2()  
}
```

} abstract methods

* An abstract class must be declared with an abstract keyword.

* It can have abstract and non-abstract methods.

* It cannot be instantiated.

* It can have constructors and static methods also.

* It can have final methods which will force the subclass not to change the body of the method.

* Abstract class cannot to be used create objects.

* We use abstract keyword to create abstract methods.

Java Packages

* Package in java used to group related class.

* Think of it as a folder in a file directory.

* We used to packages to avoid name conflicts, and to write a better maintainable code.

Packages are divided into 2 categories.

1) Built-in packages (packages from the java ^{API} ~~library~~)

2) User-defined packages (create your own package)

Built-in Packages

- * The Java API is a library of prewritten classes, that are free to use, included in the Java Development Environment.
- * The library contains components for managing input, database programming, and much more.
- * The library is divided into packages and classes. Meaning you can either import a single class (along with its methods & attributes), or a whole package that contains all the classes that belong to the specified package.

```
import package.name.class ; // import a single class  
import package.name.* ; // import the whole package
```

Import a Class

ex-
`import java.util.Scanner;`
 ↑ package ↗ class

- * java.util is a package, while Scanner is a class of the java.util package.

- * To use the scanner class, create an object of the class and use any of the available methods found in the Scanner class documentation.

* This package also contains date and time facilities, random number generator and other utility classes

User-Defined Packages

* To create your own package, java, uses a file system directory to store them.

```
└─ root
   └─ mppack
      └─ MyPackagerClass.java
```

* To create a package, use the package keyword.

```
package myPack;
class MyPackagerClass {
}
```

Advantages of package in java;

(1) Package provides unique namespace.

(2) Package provides access protection.

(3) Package provides grouping of logically related classes & interfaces so easy to maintain.

Java final keyword

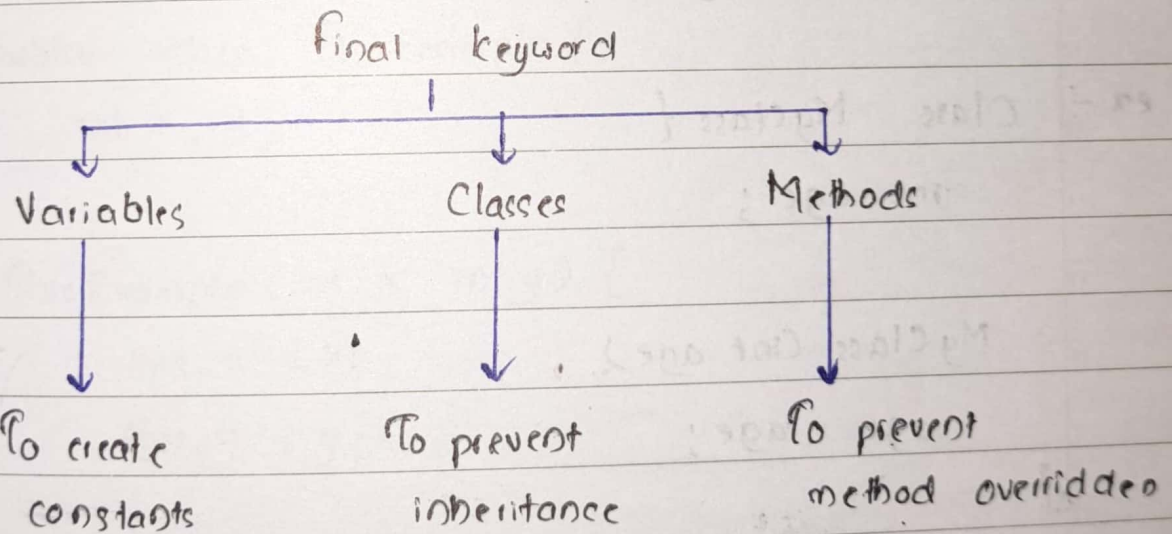
* Final keyword is used to denote constants. It can be used with variables, methods, and classes.

* Once any entity (variable, method or class) is declared final, it can be assigned only once. That is,

(1) The final variable cannot be reinitialized with another value.

(2) Final method cannot be overridden.

(3) Final class cannot be extended.



This keyword

* This keyword is used to refer to the current object inside a method or a constructor.

ex:- class Main {

Date

No

int instVar;

1 Main (int instVar) {

2 this.instVar = instVar;

3 Sysout ("this reference?" + this);

4 }

5
6 Using this for Ambiguity Variable Names

7
8 * In java, it is not allowed to use declare two or more
9 variables having the same name inside a scope
10 (class scope or method scope). However, instance variables
11 and parameters may have the same name.

12
13 ex:- class MyClass {

14 int age;

15
16 MyClass (int age)

17 age = age;

18 }

19 Using this in constructor overloading

20
21 class Complex {

22 private int a, b;

23 private Complex (int i, int j) {

24 this.a = i;

25 this.b = j;

}

IMate


```
private Complex (int i) {
    this (i, 1)
}
```

```
private Complex () {
    this (0)
}
```

Passing this as an Argument

* We can use this keyword to pass the current object as an argument to a method.

```
eg:- public class ThisExample {
    int x, y;
```

```
    ThisExample (int x, int y) {
```

```
        this.x = x;
```

```
        this.y = y;
```

```
        add (this);
```

```
    } void add (ThisExample o) {
```

```
        o.x = 2;
```

```
        o.y = 2;
```

data type

Variable

Super Keyword in Java

* It is a reference variable which is used to refer immediate parent class object.

* Whenever you create the instance of subclass, an instance of parent class is created implicitly which is referred by super reference variable.

Usage of java Super Keyword

- 1) Used to refer immediate parent class instance variable.
- 2) Used to invoke immediate parent class method.
- 3) Super () can be used to invoke immediate parent class constructor.

* We can use super keyword to access the data member or field of parent class. It is used if parent class and child class have same field.

```
ex:- Class Animal {  
    String color = "White";  
}
```

```
Class Dog extends Animal {
```

```
    String color = "Black";
```

```
    void print color(){
```

```
        Sysout (print color); output → Black
```

```
        Sysout (super.color); Output → White
```

* The super keyword is used to invoke parent class method.
It should be used if subclass contains the same method as parent class. In other words, it is used if method is overridden.

```
ex:- class Animal {  
    void eat() {  
        Sysout ("eating");  
    }  
} class Dog extends Animal {  
    void eat() { sysout ("eating bread"); }  
    void bark() { sysout ("barking"); }  
    super.eat(); print → eating  
    bark(); print → barking  
}
```

* Super keyword can also be used to invoke the parent class constructor.

```
ex:- class Animal {  
    Animal () {  
        Sysout ("animal.");  
    }  
} class Dog extends Animal {  
    Dog () {  
        Super ();  
        sysout ("dog");  
    }  
}
```


Static import in Java

* Static import in java allows importing static members of the class and use them, as they are declared in the same class.

* Many programmers think that using static import can reduce code size and allow you to freely use static field of external class without prefixing class name on that.

* Static import is a feature which provides the facility to access static members of a class directly without using class name.

* Similar to regular import statement, static import also allows wildcard * to import all static members of a class.

import	Static Import
* Access classes and interfaces without package name	* Access static members without class and interface name.
* Accessibility ^{is} for classes and interfaces.	* Accessibility is for static data members.

// Without static import ——— (import java.lang.Math.*;)

```
ex-1. Class Greets {
    public static void main (String [] args)
    {
        Sysout (Math.sqrt (4));
        Sysout (Math.pow (2,2));
        Sysout (Math.abs (6.3));
    }
}
```

// With static import ———

```
import static java.lang.Math.*;
```

```
Class Test {
```

```
    public static void Main (String [] args)
```

```
    {
        Sysout (sqrt(4));
```

```
        Sysout (pow(2,2));
```

```
        Sysout (abs(6.3));
```

```
    }
}
```

import static

java.lang.

Integer, MAX-VALUE

Integer, MIN-VALUE

sysout (MAX-VALUE)

sysout (MIN-VALUE)

Java Static Keyword

* If we want to access class members, we must first create an instance of the class. But there will be situations where we want to access class members without creating any variable.

* In those situations, we can use the static keyword in java. If we want to access class members without

creating an instance of the class, we need to declare the class members static.

```

ex:- public class Main {
    public static void myStaticMethod () {
        SysOut (" Static methods can be called without creating
        Objects" );
    }
    public void myMethod () {
        Sysout (" public methods must be called by creating
        objects");
    }
    public static void Main (String [] args) {
        myMethod (); ← X
        my StaticMethod (); ← ✓
    }

```

Static Variables

* If we declare variable static, all objects of the class share the same static variable. It is because like static methods, static variables are also associated with the class. And, we don't need to create objects of the class to access the static variable.

```

ex:- Class Test {
    static int age;
}
class Main {
    Test.age = 20;
}

```


Instance Initializer Block

* Instance initializer block is a mechanism provided by java compiler to define a group of statements common to all constructors at a single place.

* At the compilation time, compiler moves these statements at the beginning of all constructors after super.

* If static and non-static anonymous blocks are used then static anonymous block is executed only once.

```
ex: public class Main {
```

```
{
```

```
    Sysout("Non Static Anonimus block");
```

```
}
```

```
    static {
```

```
        Sysout("Static Anonymous block called");
```

```
}
```

```
    public static void main(String[] args) {
```

```
}
```

Aggregat

Aggregation in Java

* If a class have an entity reference, it is known as aggregation. Aggregation represent HAS - A relationship.

* Consider a situation, Employee object contains many information such as id, name, email etc. It contains one more objects named address, which contains its own informations such as city, state, country, zipcode etc. as given below.

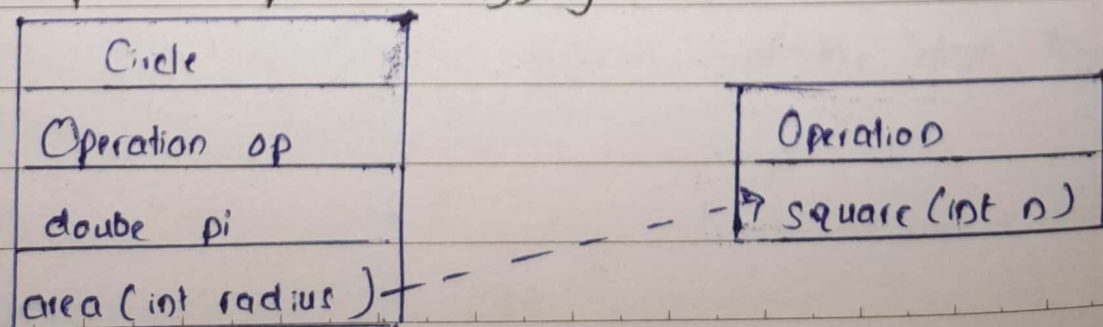
```
class Employee {
    int id;
    String name;
    class Address address; // Address is a class name
```

In such case, Employee has an entity reference address, so relationship is Employee HAS - A address.

Why use aggregation?

* For code reusability

Simple example of Aggregation



ex: Class Operation {

```
int square (int n) {  
    return n*n;  
}}
```

Class Circle {

```
Operation op; double pi = 3.14;
```

```
double area (int radius) {
```

```
    op = new Operation ();
```

```
    int square = op.square (radius); // code reusability)
```

```
    return pi * square;
```

```
}
```

```
public static void main (String [] args) {
```

```
    Circle c = new Circle ();
```

```
    double result = c.area (5);
```

```
    Sysout (result);
```

When use aggregation?

* Code reuse is also best achieved by aggregation when there is no is-a relationship.

* Inheritance should be used only if the relationship is maintained throughout the lifetime of the objects involved; otherwise, aggregation is the best choice.

Java program for call by Value Date _____ No _____

```
class Test {  
    void meth (int i, int j)  
    { i *= 2;    j /= 2;  
    }  
  
    class callByValue {  
        public static void main (String [] args) {  
            Test ob = new Test ();  
            int a = 15, b = 20;  
            Sysout ("Before call : a=" + a + " b=" + b);  
            ob.meth (a, b);  
            Sysout ("After call : a=" + a + " b=" + b);  
        }  
    }  
}
```

Call by reference

```
class Test {  
    int a, b;  
    Test (int i, int j) {  
        a = i;  
        b = j;  
    }  
    void method (Test obj) {  
        obj.a *= 2;    obj.b /= 2;  
    }  
  
    class callByRef {  
        public static void main (String [] args) {  
            Test t1 = new Test (15, 20);  
            Sysout ("Before call ob.a=" + t1.a);  
            t1.method (t1);  
            Sysout ("After " ob.b=" + t1.b);  
        }  
    }  
}
```


1 List<String> ← List is a data type.

2
3 List<String> list = new ArrayList<String>; // using with
4 list.add("Banana"); array.

5 list.add("Mango");

6 list.remove();

7 for (String fruits : list)

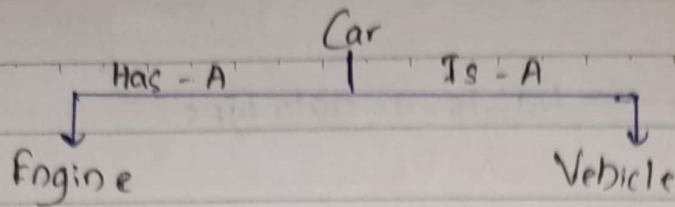
8 SysOut(fruits) → print all values

9 sysout(fruits[0]) → print only Banang.

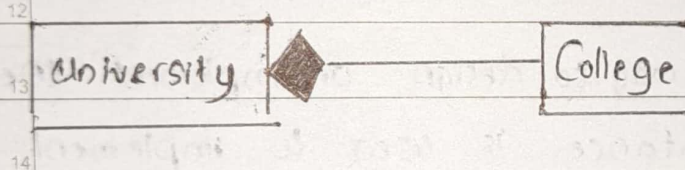
11 Composition in Java

12
13 * The composition is a way to design or implement the "has-a"
14 relationship. The inheritance is used to implement the
15 "is-a" relationship. The "has-a" relationship is used to ensure
16 the code reusability in our program. In composition, we
17 use an instance variable that refers to another object.

18
19 * The composition relationship of 2 objects is possible when
20 one object contains another object, and that object is
21 fully dependant on it. The contained object should not
22 exist without the existence of its parent object. In a
23 simple way, we can say it is a technique through which
24 we can describe the reference between 2 or more classes.
25 And for that, we use the instance variable, which
should be created before it is used.



ex -> We create a class College that contains variable (Name, address). We also create a class University that has a reference to refer to the list of colleges. A university can have more than one colleges. So, if a university is permanently closed, then all colleges within that particular university will be closed because colleges cannot exist without a uni. The relationship between the university and college is composition.



Java Polymorphism

Types of polymorphism,

- 1) Static / Compile time polymorphism (by method overloading)
- 2) Dynamic / Run time polymorphism (by method overriding)

Method overloading in Java.

* Method overloading is the way of implementing static / compile time polymorphism in Java.

* Method overloading means more than 1 method in a class with same name but different parameters.

* Parameters can be differing in types, number or orders.
Compiler resolve method call by matching method signature at compile time, that's why it is known as static or compile time polymorphism.

* Static methods cannot be overridden.

* It is also known as static binding.

Ways to implement method overloading in java.

1) Parameters differs in types.

2) Parameters differ in number.

3) Parameters differ in order.

———— methods overridden example ————

ex-1: Class Polygon {

public void render () {

sysout ("Rendering Polygon...");

}

} Class Square extends Polygon {

public void render () {

sysout ("Rendering square");

}

Class Main {

public static void main (String [] args) {

Square s1 = new Square();

s1.render();

Circle c1 = new Circle();

c1.render();

Method Overridden in Java

* If a subclass provides the method with the same signature (name and parameter) as in its super class, then subclass overrides the method of its super class.

* This process of overriding in a super class method by subclass is known as method overridden.

— method overloading example —

```
class Pattern {  
    public void display() {  
        for (int i=0; i<10; i++) {  
            sysout ("*");  
        }  
    }  
    public void display (char symbol) {  
        for (int i=0; i<10; i++) {  
            sysout (symbol);  
        }  
    }  
}
```

```
class Main {  
    public static void main (String[] args) {  
        pattern di = new Pattern();  
        di.display();  
        sysout ("ln");  
        di.display('#');  
    }  
}
```


Java Operator Overloading

Some operators behave differently with different operands.

ex:- $+$ operator is overloaded to perform numeric additions as well as string concatenation, and

x Operators like $\&$, $|$, and $!$ are overloaded for logical and bitwise operations.

The $+$ operator is used to add 2 entities. In java, $+$ operator performs 2 operations.

1) When $+$ is used with numbers (Integers or floating point numbers), it performs mathematical addition.

ex:- `int a = 5; int b = 6;`

`int sum = a + b;`

2) When we use the $+$ operator with strings, it will perform string concatenation (join 2 strings)

ex:- `String first = "Java";`

`String second = "Programming";`

`String name = first + second;`

Polymorphic Variables

x A variable is called polymorphic if it refers to different values under different conditions.

* Object variables (instance variables) represent the behavior of polymorphic variables in java.

* It is because object variables of a class can refer to objects of its classes as well as objects of its subclasses.

```
ex: public class programmingLanguage {  
    public void display () {  
        sysout ("I am programming language");  
    }  
}
```

```
class Java extends ProgrammingLanguage {  
    public void display () {  
        sysout ("I am OOP Language");  
    }  
}
```

```
class Main {  
    public static void main (String[] args) {  
        ProgrammingLanguage p1;  
        p1 = new ProgrammingLanguage();  
        p1.display();  
    }  
}
```

```
    p1 = new Java();  
    p1.display();  
}
```


Unified Modeling Language (UML)

* UML is a standardized general purpose modeling language in the field of object-oriented software engineering.

UML diagram represent 2 different views of a system model.

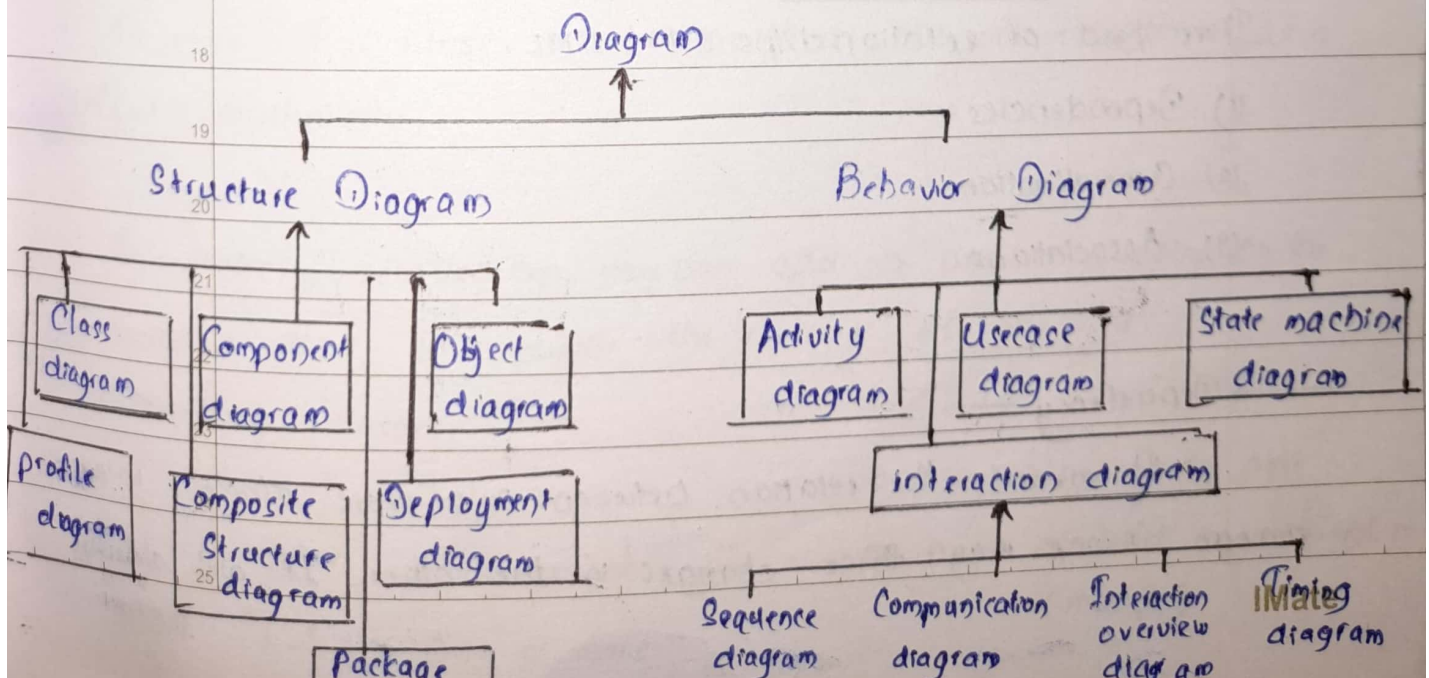
1) Static (or Structural) view

This view ~~emphas~~ emphasizes the static structure of the system using objects, attributes, operations and relationships
ex: Class diagram, Composite structure diagram

2) Dynamic (Behavioral) view

This view emphasizes the dynamic behavior of the system by showing collaborations among objects and changes to the internal state of objects.

ex: Sequence diagram, Activity diagram, State machine diagram.



Class Diagram

- * A picture of the classes in an OO system, their fields and methods, and connection between the classes that interact or inherit from each other.
- * A classic description of a set of objects that share the same attributes, operations, relationships and semantics.
- * Graphically, a class is rendered as a rectangle, usually including its name, attributes and operations in separate, designated compartments.

class Name
attributes
Operations

Attributes can be,

* + public

* - private

* # protected

* / derived

Relationships

Three kind of relationships in UML.

(1) Dependencies

(2) Generalizations

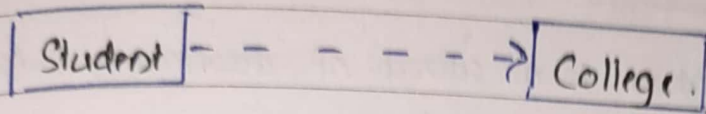
(3) Associations

Dependency

* It means the relation between 2 or more classes in which a change in one may force changes in the other. It will always

Date _____ No _____

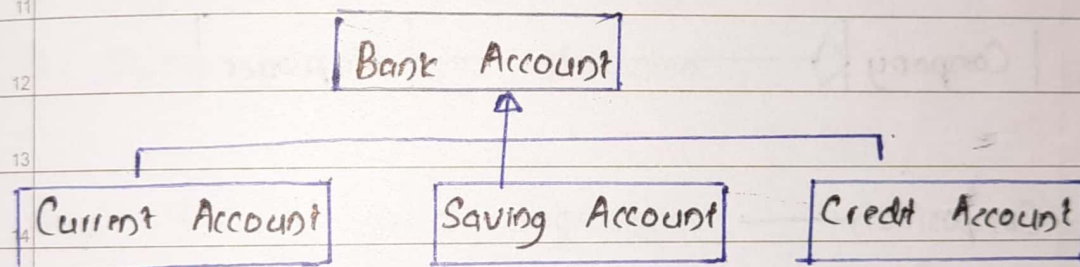
create a weaker relationship. Dependency indicates that one class depends on another.



Student has a dependency on College.

— Generalization —

* This helps to a subclass to its superclass. A sub-class is inherited from its superclass. Generalization can't be used to model interface implementation. Class diagram allows inheriting from multiple superclasses.



— Multiplicity —

* It defines a specific range of allowable instances of attributes. In case if a range is not specified, one is considered as a default multiplicity.

In multiplicity relationship, you can add a number directly to the associated line to indicate the number of objects in the corresponding classes.

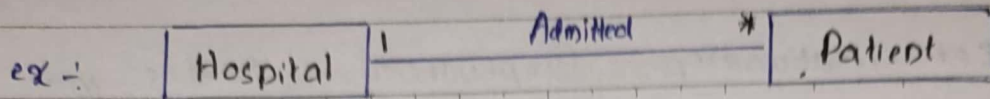
1..1 - Only one

0..* - Zero or more

1..* - One or more

0..1 - No or only one

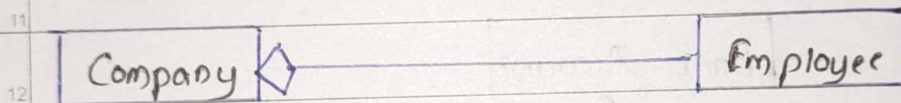
m..n - at least m, at most n
(m ≤ n) IMate



Aggregation

* An aggregation is a subset of association, which represents a relationship. It is more specific than association. It defines a part-whole or part-of relationship. In this kind of relationship, the child class can exist independently of its parent class.

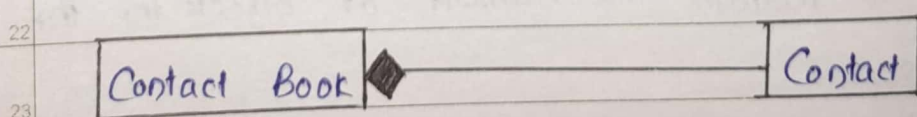
ex: The company encompasses a no of employees, and even if one employee resigns, the company still exists.



Composition

* It is a subset of aggregation. It portrays the dependency between the parent and its child, which means if one part is deleted, then the other part also gets discarded. It represents a whole-part relationship.

ex: A contact book consists of multiple contacts, and if delete the contact book, all the contacts will be lost.



Object Diagrams

* Object diagrams are derived from class diagrams, so object diagrams are dependent upon class diagrams.

* The purpose of a diagram should be understood clearly to implement it practically.

* Understand object and behavior and their relationship from practical perspective :

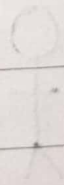
(1) Forward & reverse engineering

(2) Object relationships of a system

(3) Static view of an interaction

~~***~~ Where to use object diagrams :-

* Object diagrams can be imagined as the snapshot of a running system at a particular moment.



UML - Interaction Diagram

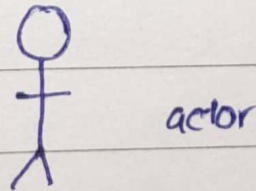
* This interactive behavior is represented by in UML by 2 diagrams known as sequence diagram and Collaboration diagram.

Sequence Diagram

* From the name, it is clear that the diagram deals with some sequences, which are the sequence of messages flowing from one object to another.

Basic Symbols and Components

1) Actor - It is represent a type of role where it interacts with the system n its objects. It is important to note here that an actor is always outside the scope of the system we aim to model using UML diagram.



2) Lifeline Symbol - This dashed vertical line shows the sequential events that occur to an object during the charted process. Lifelines may begin with a labeled rectangle shape or an actor symbol.

class Name
: User

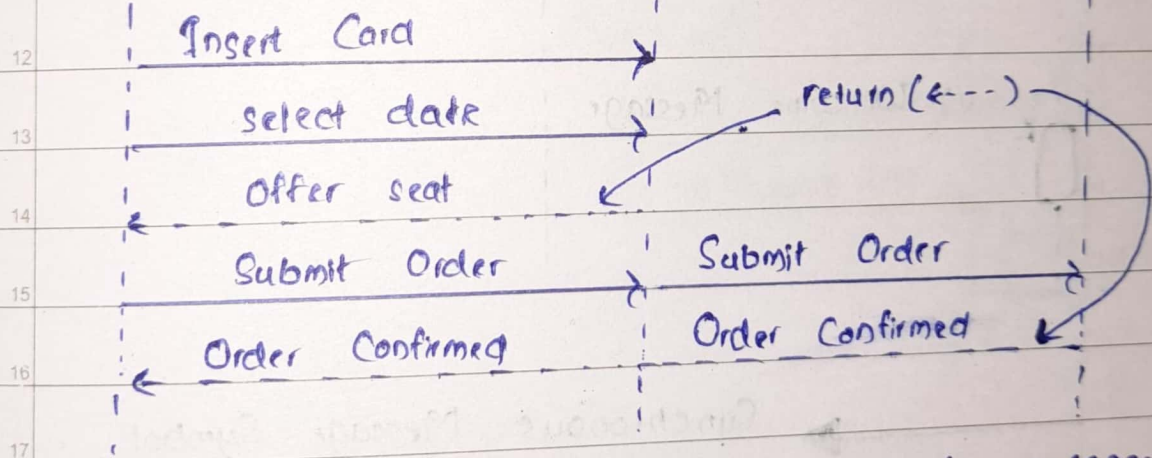
x: Class 1

Here x is the object or instance name class 1 is the class name

ex :- user

x: Theater

S: Server



* Communication between objects is represented by messages. It is appear in a sequential order on the lifeline.

Messages can be broadly classified into 2 categories.

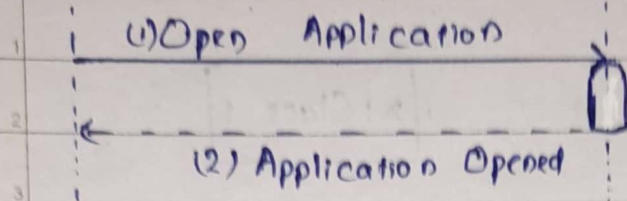
1) Synchronous Messages - A synchronous message waits for a reply before the interaction can move forward. The sender waits until the receiver has completed the processing of the message.

User

Device

Date

No



(2) Asynchronous Message - It does not wait for a reply from the receiver. The interaction moves forward irrespective of the receiver processing the previous message or not. We use a lined arrow head to represent an asynchronous message.



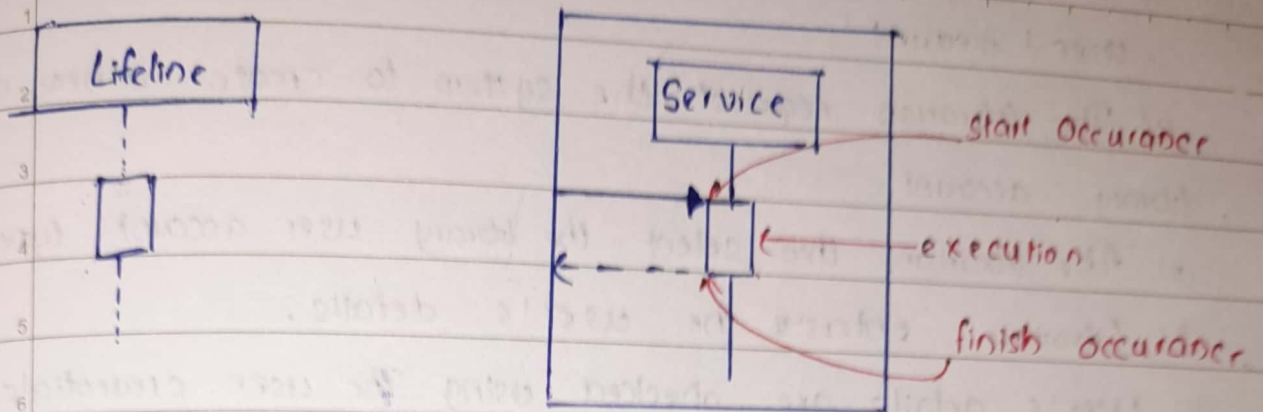
17 → Synchronous Message Symbol

19 → Asynchronous Message Symbol

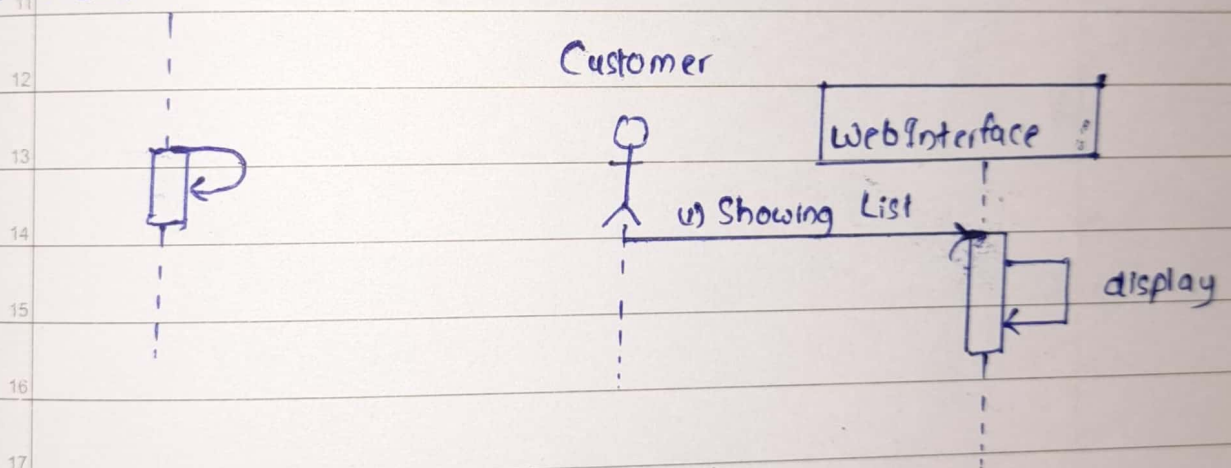
(3) Activation

* It is represented by a thin rectangle on the lifeline. It describes that time period in which an operation is performed by an element, such that the top and the bottom of the rectangle is associated with the initiation and the

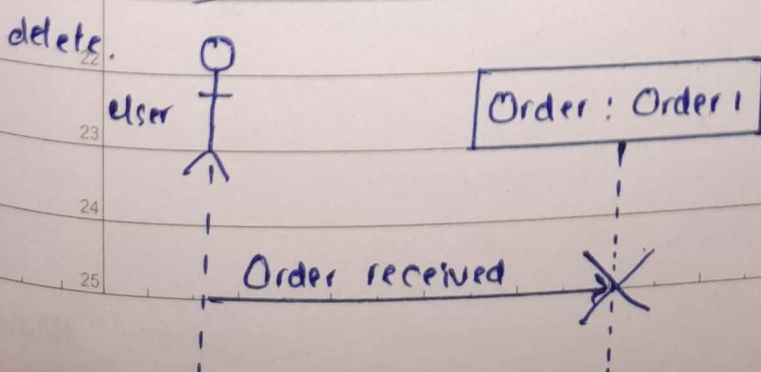
completion time, each respectively



(4) Self Message - It describes a communication particularly between the lifelines of an interaction that represents a message of a same lifeline has been invoked.



(5) Delete Message - We use a delete message to delete an object. When an object is ~~del~~ deallocated memory or is destroyed within the system we use



Steps that occurs in the use case named 'Create new library user account'.

- 1) The librarian request the system to create a new online library account.
- 2) The librarian then select the library user account type.
- 3) Librarian enters the user's details.
- 4) User's details are checked using the user credentials database.
- 5) The new library user account is created.
- 6) A summary of the new account's details is then emailed to the user.